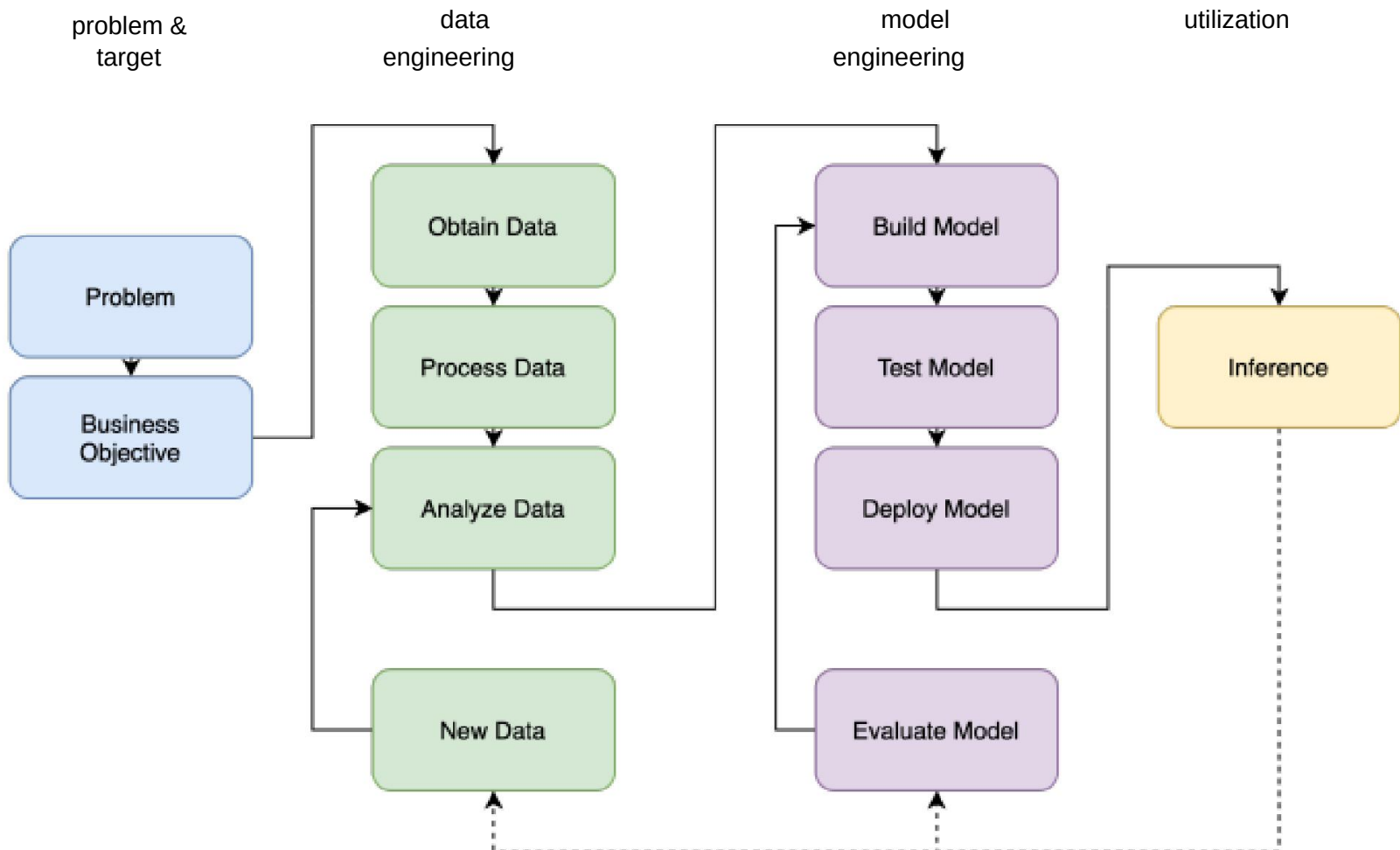


Cross Validation & Dataset Preparation

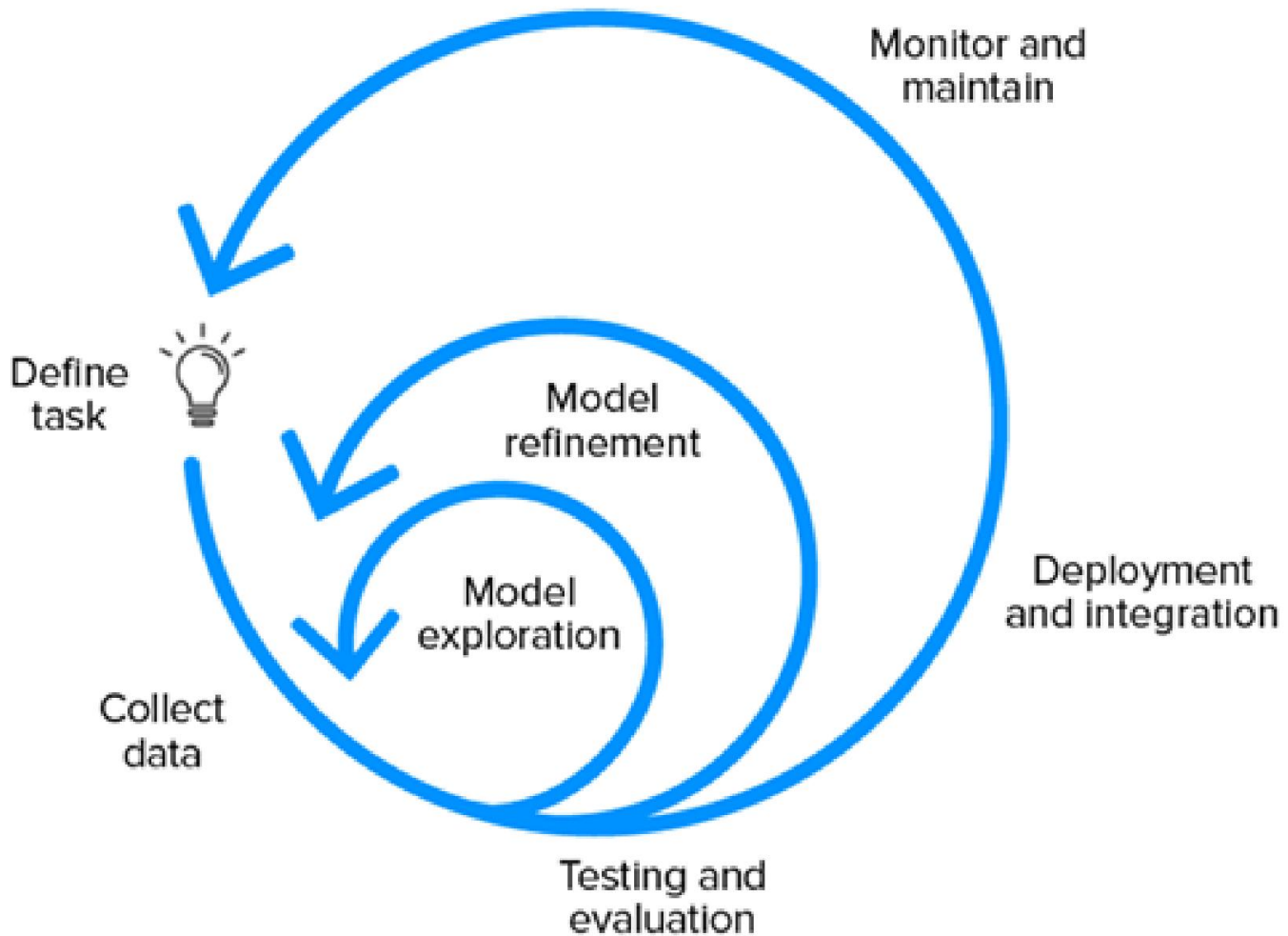
detail

- lifecycle of ML model & data
- Model fit & overfitting
- Cross-validation
- Cross-validation types and data preparation
- Cross-validation examples

lifecycle of ML model & data (1)



lifecycle of ML model & data (2)



Problems with model fit

- Common goodness-of-fit measures in linear models: R^2 statistic

$$R^2 = 1 - \frac{\sum(y_i - f_i)^2}{\sum(y_i - \bar{y})^2}$$

← Model error ← Variance in the dependent variable

$$= 1 - \frac{(\text{실제 } Y \text{와 추정된 } Y \text{의 차이를 제공한 값의 합계})}{(\text{실제 } Y \text{와 } Y \text{평균값간 차이를 제공한 값의 합계})}$$

- the ratio of error in a model over the total variance in the dependent variable.
- the lower the error, the higher the R^2 value.

- example

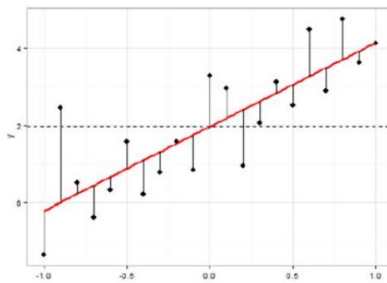
(case 1) $\sum(y_i - f_i)^2 = 18.568$

$$\sum(y_i - \bar{y})^2 = 55.001$$

$$R^2 = 1 - \frac{18.568}{55.001}$$

$$R^2 = 0.6624$$

A decent model fit!



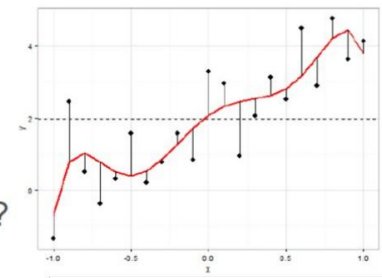
(case 2) $\sum(y_i - f_i)^2 = 15.276$

$$\sum(y_i - \bar{y})^2 = 55.001$$

$$R^2 = 1 - \frac{15.276}{55.001}$$

$$R^2 = 0.72$$

Is this a better model?
No, **overfitting!**



Overfitting

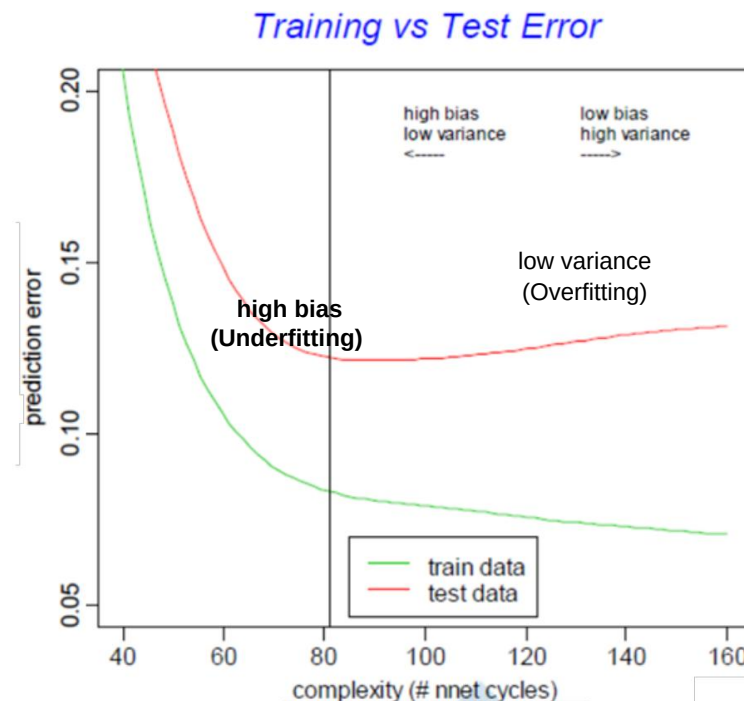
- Error on the dataset used to fit the model

can be misleading

- Doesn't predict future performance.
- Too much complexity can diminish model's accuracy on future data.
- Sometimes called the Bias-Variance Tradeoff.

- Bias-Variance Tradeoff

- Low “bias”:
 - The model fits well.
 - ie, the model value is close to the data's expected value.
- High “Variance”:
 - Model more likely to make a wrong prediction.



Cross-validation (1)

- Statistical methods for evaluating the performance (or accuracy) of a prediction model
 - In particular, used to prevent overfitting of predictive models when the amount of data is limited
- Basic method
 - Configure a fixed number of folds (or partitions) – Run analysis on each fold, then calculate by averaging the total error estimates (or performance metrics)
- Select the prediction model of the algorithm most suitable for the problem domain
 - Compare models constructed with selectable algorithms to make a selection
 - Appropriate utilization of the dataset obtained from model construction and performance verification
 - Solutions to the following problems
 - An error implying that training data represents the real world
 - In other words, the training data is a limiting factor, representing only a tiny fraction of all existing data.
 - To evaluate the actual performance of the model, a test dataset that the model has never encountered is required.
 - If the test dataset, which is part of the acquired training data, is not used for model construction (training), test
There is a risk of loss of information included in the dataset.

Cross-validation (2)

- Evaluation of performance through cross-validation (CV) –

Mean performance • The arithmetic mean of

performance metrics (accuracy, recall, etc.) obtained from multiple folds • A higher mean value indicates a better model

– Variance or standard deviation of performance

- Individual performances measured across multiple folds are of the degree of variance
- Stability indicators of model performance based on data per fold
- Model stability that demonstrates consistent performance with changes in data (high variance indicates overfitting or instability of the model)

- Appropriate Bias-Variance Trade-off

– If the number of splits (k) is large, the training data increases and the bias decreases, but fold

As the similarity between models increases, the variance increases.

– Considering the balance between the two indicators, $k=5$ or $k=10$ CV is usually used.

- In other words, CV aims to identify models with high mean values and low variance performance.

A plan to

Cross-validation (3)

- application
 - model evaluation
 - lift curve or gains chart (ROC, AUC, etc.)
 - train/test method
 - model selection
 - Selection of parameters
 - # of variables in a regression
 - # of neural net nodes
 - # of leaves in a tree
 - a model showing the lowest CV error rate.

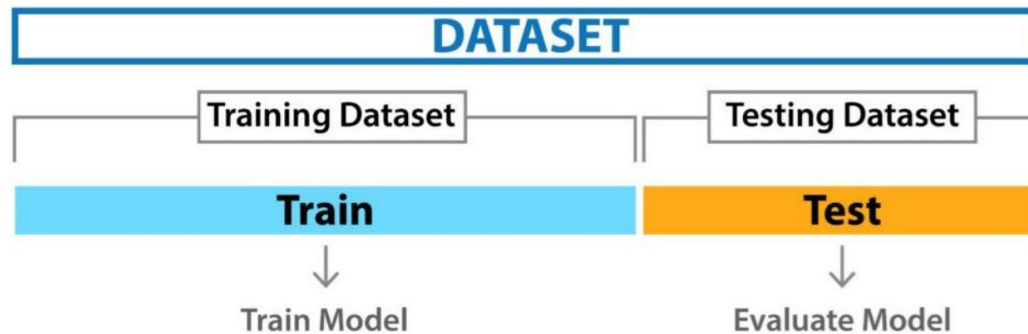
Data preparation by cross-validation type

- non-exhaustive methods
 - holdout
 - shuffle split
 - k-fold
 - stratified k-fold
 - group k-fold
- exhaustive methods
 - leave-1-out cross validation
 - leave-p-out cross validation
- time series split

holdout method (1)

- The basic method of dividing the entire training dataset into two parts (training data & test data)

In method



- After training the model with training data, evaluate it with test data
 - If necessary, extract validation data from the training data and use it.
- When there is a sufficient amount of data and training materials can be constructed through sampling
- Generally, the size of the training data is set to at least twice the size of the test data.
 - 70:30, 75:25, or 80:20, etc.
- Disadvantage that the model's performance evaluation results vary depending on the evaluation data
 - Repetitive learning and evaluation are required

holdout method (2)

- Randomly shuffle the entire training dataset before splitting

Depending on the combination of data points in the model 's training, different results are possible, and

This causes instability in model performance

– In other words, it is difficult for the selected training dataset to represent the entire dataset.

- If the dataset size is not large, the test data may contain important information

possibility

– Since the test dataset is not used for model construction, there is a possibility of losing the relevant information

- The holdout method is used when the dataset is very large, time crunch, or initial (prototype)

Useful for model building

holdout method (3)

- Types of holdout methods (examples)

(1) Basic holdout

- Randomly shuffle the dataset and split it into a training set and a test set (0.75:0.25)

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=111)
```

(2) 3-way holdout (train, val, test) • Split the

dataset into 3 (training, validation, evaluation) (6:2:2)

```
X_temp, X_test, y_temp, y_test = train_test_split(X, y, test_size=0.2, random_state=111)  
X_train, X_val, y_train, y_val = train_test_split(X_temp, y_temp, test_size=0.25, random_state=111)
```

(3) Stratified holdout

- Split the training and test sets with equal class rates

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y, random_state=111)
```

shuffle split (1)

- Random split cross-validation

- or repeated holdout, repeated random ramping

- Repeat holdout several times

- Since evaluation data is obtained through random sampling from training materials, each sampling

As the training and evaluation data change each time, the values of the model's evaluation scales also change.

- possibly with stratification or stratified shuffle split

- The final estimate of the performance metric uses the average value based on the number of trials.

- Increasing the number of iterations reduces the sample variance, leading to more precise estimates and a new model

Securing Thunderbolt

- Control how many times a specific data sample is included in the training or evaluation data

It is impossible

shuffle split (2)

- Python Example

- shuffle split

```
from sklearn.model_selection import ShuffleSplit ss =
ShuffleSplit(n_splits=3, test_size=0.3, random_state=0)
X = np.array([0, 0, 0, 1, 1, 1])
for train_index, test_index in ss.split(X):
    print("%s %s" % (train_index, test_index))
```

```
[1 3 0 4] [5 2] [4
0 2 5] [1 3] [1 2
4 0] [3 5]
```

- stratified shuffle split

```
from sklearn.model_selection import StratifiedShuffleSplit ss =
StratifiedShuffleSplit(n_splits=5, test_size=0.3, random_state=0)
X = np.array([0, 1, 2, 3, 4, 5])
y = np.array([0, 0, 0, 1, 1, 1])
for train_index, test_index in ss.split(X, y):
    print("%s %s" % (train_index, test_index))
```

```
[2 5 1 3] [0 4] [0
4 3 1] [2 5] [0 4
3 1] [2 5] [5 4 1
2] [0 3] [1 5 2 4]
[0 3]
```

k-fold & stratified k-fold (1)

- k-fold CV (cross validation)

- Step 1: Randomly divide the entire dataset into k subsets (folds) of the same size.
- Step 2: Use each subset for testing in turn, and the rest for training.

Iteration 1	Test	Train	Train	Train	Train
Iteration 2	Train	Test	Train	Train	Train
Iteration 3	Train	Train	Test	Train	Train
Iteration 4	Train	Train	Train	Test	Train
Iteration 5	Train	Train	Train	Train	Test

- Concerns about class overlap in the test dataset

- Avoid class overlap: Process stratification per fold if necessary

- The final estimate of the performance metric is the average value of the k-times training-test error.

- standard method: stratified 10-fold CV

- Why 10? : Experimental results (theoretical basis?)
- Stratified 10-fold CV is better.

- Stratification has the effect of reducing the variance of performance estimates

k-fold & stratified k-fold (2)

- Possibility of class overlap in the fold used as test data → Utilize stratified k-fold CV

- Python examples

- k-fold CV

```
import numpy as np
from sklearn.model_selection import KFold kf =
KFold(n_splits=3)
X = np.array([0, 0, 0, 0, 1, 1, 1, 1, 1])
for train, test in kf.split(X):
    print("%s %s" % (train, test))
```

```
[3 4 5 6 7 8] [0 1 2] [0
1 2 6 7 8] [3 4 5] [0 1 2
3 4 5] [6 7 8]
```

- stratified k-fold CV

```
import numpy as np
from sklearn.model_selection import StratifiedKFold skf =
StratifiedKFold(n_splits=3)
X = np.ones(9)
y = [0, 0, 0, 1, 1, 1, 2, 2, 2]
for train, test in skf.split(X, y):
    print("%s %s" % (train, test))
```

```
[1 2 4 5 7 8] [0 3 6] [0
2 3 5 6 8] [1 4 7] [0 1 3
4 6 7] [2 5 8]
```

group k-fold

- Separate the training dataset and test dataset so that they are not included in the same group.
 - Used when making predictions on new data that differs from existing data • (e.g.) Predicting outcomes by diagnosing a new patient using existing patient data
- Each record in the data requires a group attribute value

- (example)

```
X = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] y =
["a", "b", "b", "b", "c", "c", "c", "d", "d", "d"] groups = [1,
1, 1, 2, 2, 2, 3, 3, 3, 3]
```

- Python examples

```
from sklearn.model_selection import GroupKFold gkf =
GroupKFold(n_splits=3)
X = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11] y =
["a", "b", "b", "b", "c", "c", "c", "d", "d", "d", "d"] groups = [1,
1, 1, 2, 2, 2, 3, 3, 3, 3, 3] for train, test in
gkf.split(X, y, groups=groups): print("%s %s" % (train,
test))
```

```
[0 1 2 3 4 5] [6 7 8 9 10] [0 1
2 6 7 8 9 10] [3 4 5] [3 4 5 6
7 8 9 10] [0 1 2]
```

leave-one-out(LOO)

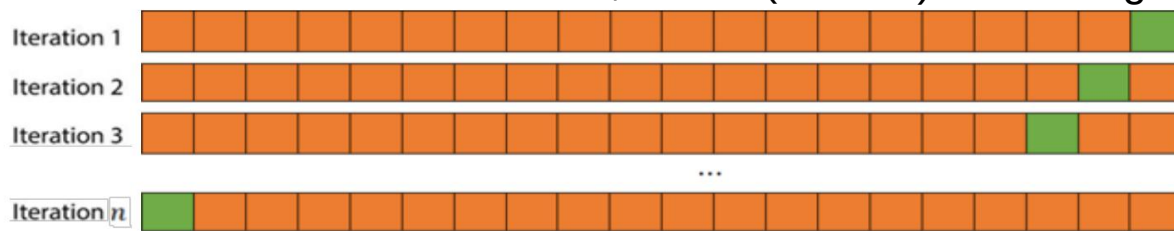
- exhaustive CV method

- all possible ways to divide the original sample into a training and a validation (or test) set

- a particular form of cross-validation

- # of folds = # of whole dataset (training + test)

ie, a model has to be built n times, where $(n - 1)$ for training & 1 for test



- makes maximum use of the data

- no random subsampling involved, computationally expensive

- Python examples

```
from sklearn.model_selection import LeaveOneOut
loo = LeaveOneOut()
X = [1, 2, 3, 4, 5]
for train, test in loo.split(X):
    print("%s %s" % (train, test))
```

```
[1 2 3 4] [0]
[0 2 3 4] [1]
[0 1 3 4] [2]
[0 1 2 4] [3]
[0 1 2 3] [4]
```

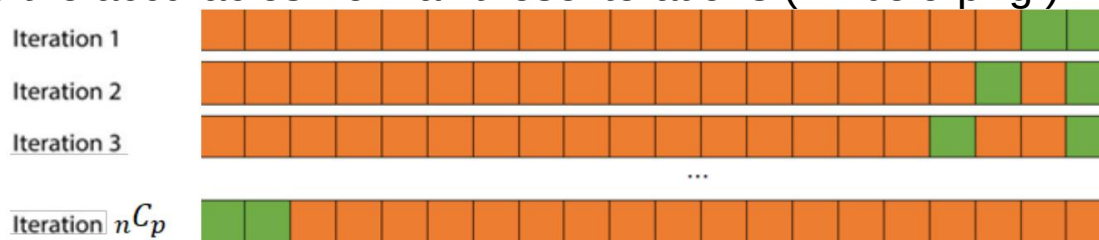
leave-out (LPO)

- an exhaustive method

- train the model on every possible combination of data points

- process

- p of points out from the total number of data points in dataset(). n
 - a model trained with $(n-p)$ points, and tested with data points. p
 - repeat the process for all the possible combinations $\# \binom{n}{p}$
 - average the accuracies from all these iterations (=2 below fig.)



- higher value for p $\rightarrow$ much more # of combinations

- the method gets a lot more exhaustive.

- Python examples

```
from sklearn.model_selection import LeavePOut
lpo = LeavePOut(p=2)
X = [1, 2, 3, 4, 5]
for train, test in lpo.split(X):
    print("%s %s" % (train, test))
```

```
[2 3 4] [0 1]
[1 3 4] [0 2]
[1 2 4] [0 3]
[1 2 3] [0 4]
[0 3 4] [1 2]
[0 2 4] [1 3]
[0 2 3] [1 4]
[0 1 4] [2 3]
[0 1 3] [2 4]
[0 1 2] [3 4]
```

time series split (1)

- or rolling cross method • basic

method (time series cross-test)

– Create folds in chronological order

- For time-series data spanning n years, when constructing n folds per year – iteration 1: training [1], test [2] – iteration 2: training [1 2], test [3]

...

– iteration n : training [1 2 3 ... $(n-1)$], test [n]

- In time-series data, shuffling risks disturbing time-order information – that is, cross-validation methods such as k -folding are not allowed for time-series data.

- Python example

```
from sklearn.model_selection import TimeSeriesSplit tscv
= TimeSeriesSplit(n_splits=3)
X = [1, 2, 3, 4, 5]
for train, test in tscv.split(X):
    print("%s %s" % (train, test))
```

```
[0 1] [2]
[0 1 2] [3]
[0 1 2 3] [4]
```



time series split (2)

- sliding time series split



- Python examples

```
from sklearn.model_selection import TimeSeriesSplit
import numpy as np
tscv = TimeSeriesSplit(n_splits=3, max_train_size=3, test_size=1)
data = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90, 100])
for train, test in tscv.split(data):
    print("%s %s" % (train, test))
```

```
[4 5 6] [7]
[5 6 7] [8]
[6 7 8] [9]
```

time series split (3)

- Time series split with a gap (interval)



- Python examples

```
import numpy as np
from sklearn.model_selection import TimeSeriesSplit tscv
= TimeSeriesSplit(n_splits=5, gap=5, max_train_size=5, test_size=2)
X = np.arange(0, 100)
for train, test in tscv.split(X):
    print("%s %s" % (train, test))
```

```
[80 81 82 83 84] [90 91] [82
83 84 85 86] [92 93] [84 85
86 87 88] [94 95] [86 87 88
89 90] [96 97] [88 89 90 91
92] [98 99]
```

CV Case (1)

- stratified k-fold CV (cv=3)
 - bank_churners dataset (10,127 samples)

```
import numpy as np
import pandas as pd
from sklearn.model_selection import KFold
df = pd.read_csv('bank_churners.csv')
kf = KFold(n_splits=3); k = 1
for train, test in kf.split(df):
    print("(%s) train data(length: %s, - index: %s)" % (k, len(train), train))
    print("(%s) test data (length: %s, - index: %s)" % (k, len(test), test)); k+=1
    df_train = df.iloc[train]
    df_test = df.iloc[test]
```

```
(1) train data(length: 6751, - index: [ 3376 3377 3378 ... 10124 10125 10126])
(1) test data (length: 3376, - index: [ 0 1 2 ... 3373 3374 3375])
(2) train data(length: 6751, - index: [ 0 ... 10124 10125 10126])
2 (2) test data (length: 3376, - index: [3376 3377 3378 ... 6749 6750 6751])
(3) train data(length: 6752, - index: [ 0 1 2 ... 6749 6750 6751])
(3) test data (length: 3375, - index: [ 6752 6753 6754 ... 10124 10125 10126])
```

- fold 1 : 3,376 samples
 - fold 2 : 3,376 samples
 - fold 3 : 3,375 samples

CV Case (2)

- stratified k-fold CV (cv=5) –

bank_churners dataset (500 samples from original)

```
import numpy as np ; import pandas as pd from
sklearn.tree import DecisionTreeClassifier from
sklearn.model_selection import StratifiedKFold, cross_validate df=
pd.read_csv('bank_churners_small.csv')
X= df.drop(columns=['Attrition_Flag']) y =
df['Attrition_Flag'] model
= DecisionTreeClassifier(random_state=156) skf =
StratifiedKFold(n_splits=5, shuffle=True, random_state=42) scoring =
['accuracy', 'precision', 'recall'] # scoring = ['accuracy', 'precision_macro', 'recall_macro'] results = cross_validate(model,
X, y, cv=skf, scoring=scoring) # k-fold CV (cv=10) print('* results\n', results) ; print('- Average
Accuracy = ', np.mean(results['test_accuracy'])) print('- Average Precision = ',
np.mean(results['test_precision'])) print('- Average Recall = ',
np.mean(results['test_recall']))
```

```
*
Result {'fit_time': array([0.00615382, 0.00324464, 0.00337577, 0.00291538, 0.0032866 ]),
'score_time': array([0.0091567, 0.00664282, 0.006042, 0.00649714, 0.00637579]), 'test_accuracy':
array([0.98, 0.97, 0.97, 0.99, 0.96]), 'test_precision':
array([0.88888889, 0.75, 0.875, 0.90909091, 0.8]), 'test_recall': array([0.88888889,
1., 0.77777778, 1., 0.8])} - Average Accuracy = 0.974 - Average
Precision =
0.8445959595959597 - Average Recall =
0.8933333333333333
```

CV Case (3)

- k-fold CV (cv=10): bank_churners dataset (500 samples from original)

```
import numpy as np ; import pandas as pd
from sklearn.model_selection import cross_validate ; from sklearn.tree import DecisionTreeClassifier
df= pd.read_csv('bank_churners_small.csv')
X= df[['Customer_Age', 'Gender', 'Dependent_count', 'Income_Category', 'Card_Category',
      'Months_on_book', 'Total_Relationship_Count', 'Contacts_Count_12_mon', 'Credit_Limit',
      'Total_Revolving_Bal', 'Avg_Open_To_Buy', 'Total_Trans_Amt', 'Total_Amt_Chng_Q4_Q1']]
y = df['Attrition_Flag']
model = DecisionTreeClassifier(random_state=156)
scoring = ['accuracy', 'precision', 'recall'] # scoring = ['accuracy', 'precision_macro', 'recall_macro']
results = cross_validate(model, X, y, cv=10, scoring=scoring) # k-fold CV (cv=10)
print('* Result\n', results) ; print('- Average accuracy = ', np.mean(results['test_accuracy']))
print('- Precision average = ', np.mean(results['test_precision']))
print('- Average recall = ', np.mean(results['test_recall']))
```

```
* result
{'fit_time': array([0.00334978, 0.00424814, 0.003901, ..., 0.00310636, 0.00319529]),
 'score_time': array([0.0083611, 0.00785875, 0.00717664, ..., 0.00585079, 0.00603795]),
 'test_accuracy': array([0.96, 1., 1., 0.96, 0.98, 0.98, 0.98, 0.96, 1., 0.96]),
 'test_precision': array([0.66666667, 1., 1., 0.71428571, 0.83333333, 0.83333333, 1., 0.8, 1., 1.]),
 'test_recall': array([1., 1., 1., 0.8, 0.8, 1., 1., 0.6, 1., 1.])}
- Accuracy average = 0.9780000000000001
- Average precision = 0.8847619047619049
- Average recall = 0.9199999999999999
```

CV Case (4)

- stratified k-fold CV (cv=10) –

bank_churners dataset (500 samples from original)

```
import numpy as np ; import pandas as pd from
sklearn.tree import DecisionTreeClassifier from
sklearn.model_selection import StratifiedKFold, cross_val_score df=
pd.read_csv('bank_churners_small.csv')
X= df[['Customer_Age', 'Gender', 'Dependent_count', 'Income_Category', 'Card_Category',
      'Months_on_book', 'Total_Relationship_Count', 'Contacts_Count_12_mon', 'Credit_Limit',
      'Total_Revolving_Bal', 'Avg_Open_To_Buy', 'Total_Trans_Amt', 'Total_Amt_Chng_Q4_Q1']] y =
df['Attrition_Flag'] model
= DecisionTreeClassifier(random_state=42) skf =
StratifiedKFold(n_splits=10, shuffle=True, random_state=42) scores =
cross_val_score( model,
```

- Accuracy per fold: [0.98 0.98 0.98 0.98 0.96 1. 1. 0.98 0.94 1.] - Average
accuracy: 0.9800

CV Case (5)

- stratified k-fold CV (cv=5) –

bank_churners dataset (500 samples from original)

```
import numpy as np ; import pandas as pd from
sklearn.tree import DecisionTreeClassifier from
sklearn.model_selection import StratifiedKFold, cross_validate df=
pd.read_csv('bank_churners_small.csv')
X= df.drop(columns=['Attrition_Flag']) y =
df['Attrition_Flag'] model
= DecisionTreeClassifier(random_state=156) skf =
StratifiedKFold(n_splits=5, shuffle=True, random_state=42) scoring =
['accuracy', 'precision', 'recall'] # scoring = ['accuracy', 'precision_macro', 'recall_macro'] results = cross_validate(model,
X, y, cv=skf, scoring=scoring) # k-fold CV (cv=10) print('* results\n', results) ; print('- Average
Accuracy = ', np.mean(results['test_accuracy'])) print('- Average Precision = ',
np.mean(results['test_precision'])) print('- Average Recall = ',
np.mean(results['test_recall']))
```

```
*
Result {'fit_time': array([0.00615382, 0.00324464, 0.00337577, 0.00291538, 0.0032866 ]),
'score_time': array([0.0091567, 0.00664282, 0.006042, 0.00649714, 0.00637579]), 'test_accuracy':
array([0.98, 0.97, 0.97, 0.99, 0.96]), 'test_precision':
array([0.88888889, 0.75, 0.875, 0.90909091, 0.8]), 'test_recall': array([0.88888889,
1., 0.77777778, 1., 0.8])} - Average Accuracy = 0.974 - Average
Precision =
0.8445959595959597 - Average Recall =
0.8933333333333333
```

CV Case (6)

- leave-one-out(LOO) CV –

bank_churners dataset (500 samples from original)

```
import numpy as np
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import LeaveOneOut, cross_val_score
df = pd.read_csv('bank_churners_small.csv')
X = df[['Customer_Age', 'Gender', 'Dependent_count', 'Income_Category', 'Card_Category',
        'Months_on_book', 'Total_Relationship_Count', 'Contacts_Count_12_mon', 'Credit_Limit',
        'Total_Revolving_Bal', 'Avg_Open_To_Buy', 'Total_Trans_Amt', 'Total_Amt_Chng_Q4_Q1']]
y = df['Attrition_Flag']
model = DecisionTreeClassifier(random_state=42)
loo = LeaveOneOut()
scores = cross_val_score(model,
```

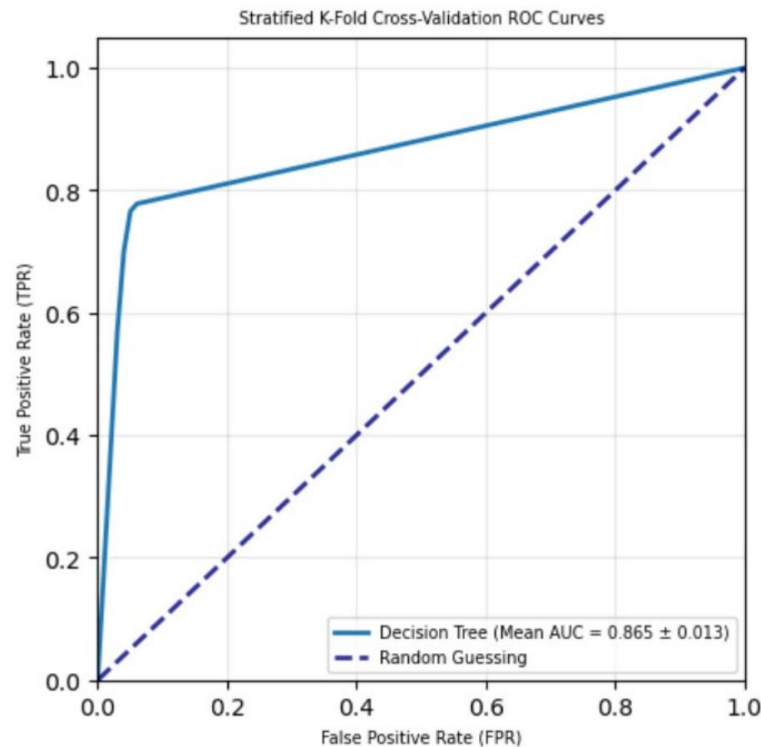
- Accuracy per fold: [1. 1. 1. 1. 0. 1. ... , 1. 1. 1. 1.] - Average accuracy: 0.9880

Model performance through k-fold (1)

- example

- dataset: bank_churners.csv –
- model: DecisionTreeClassifier
- Stratified 10-fold CV

- ROC Curve



Model performance through k-fold (2)

- example

- dataset: bank_churners.csv –

- model: LogisticRegression vs. DecisionTreeClassifier

- 10-fold CV

- ROC Curve

